Historia de Kubernetes

La historia de **Kubernetes (K8s)** es el relato de cómo una tecnología secreta de Google se transformó en el estándar absoluto para la infraestructura en la nube. [1]

1. El origen secreto: El sistema Borg (2003–2013)

Mucho antes de que el mundo hablara de "la nube", Google ya lidiaba con un problema masivo: cómo ejecutar millones de contenedores diariamente para sostener servicios como el buscador, Gmail y YouTube. [1, 2]

Para resolverlo, desarrollaron internamente **Borg** (y más tarde un sucesor llamado **Omega**). Borg era un gestor de clústeres ultra-eficiente, pero ultra-secreto y tan fuertemente acoplado a la infraestructura de Google que era imposible de usar fuera de ella. [1, 2, 3, 4]

2. El nacimiento de "Project Seven" (2014)

En 2013, tecnologías como Docker popularizaron el uso de contenedores en toda la industria. Viendo esto, los ingenieros de Google Joe Beda, Brendan Burns y Craig McLuckie (junto a Brian Grant y Tim Hockin) decidieron reescribir las lecciones aprendidas de Borg desde cero y en lenguaje Java/Go para el público general. [1, 2, 3]

El nombre clave original del proyecto dentro de Google fue **Project Seven** (Proyecto Siete). Esto fue una referencia directa al personaje *Siete de Nueve* de Star Trek, que es un "Borg" amigable. Es debido a este secreto que el timón del **logotipo oficial de Kubernetes tiene exactamente 7 radios**. [1]

3. El anuncio y la estrategia de código abierto (2014)

Google anunció oficialmente Kubernetes a mediados de **2014** en la conferencia DockerCon. [1, 2]

La decisión de liberarlo como código abierto fue un movimiento estratégico clave de Google para competir contra el dominio que tenía Amazon Web Services (AWS) en el mercado cloud. Al estandarizar la orquestación, hacían que las aplicaciones fueran portables de una nube a otra, impidiendo el bloqueo (*lock-in*) de AWS. [1]

4. La era CNCF y la dominación del mercado (2015–2018)

- **21 de julio de 2015:** Se lanza oficialmente **Kubernetes v1.0**. [1, 2]
- **La donación:** Para garantizar la neutralidad del proyecto y que sus competidores (como Microsoft, RedHat o IBM) también colaboraran,

Google donó el código a la recién creada **Cloud Native Computing Foundation (CNCF)** bajo la tutela de la Linux Foundation. [1, 2]

- **El fin de la guerra de orquestadores (2017):** Durante un tiempo, K8s compitió contra otras herramientas como *Docker Swarm* y *Apache Mesos*. Sin embargo, la flexibilidad y el inmenso apoyo comunitario de Kubernetes hicieron que ganara la batalla. En 2017, la propia Docker claudicó e integró Kubernetes de manera nativa en su plataforma.
- **Graduación (2018):** Se convirtió en el primer proyecto en graduarse oficialmente de la CNCF. [1]

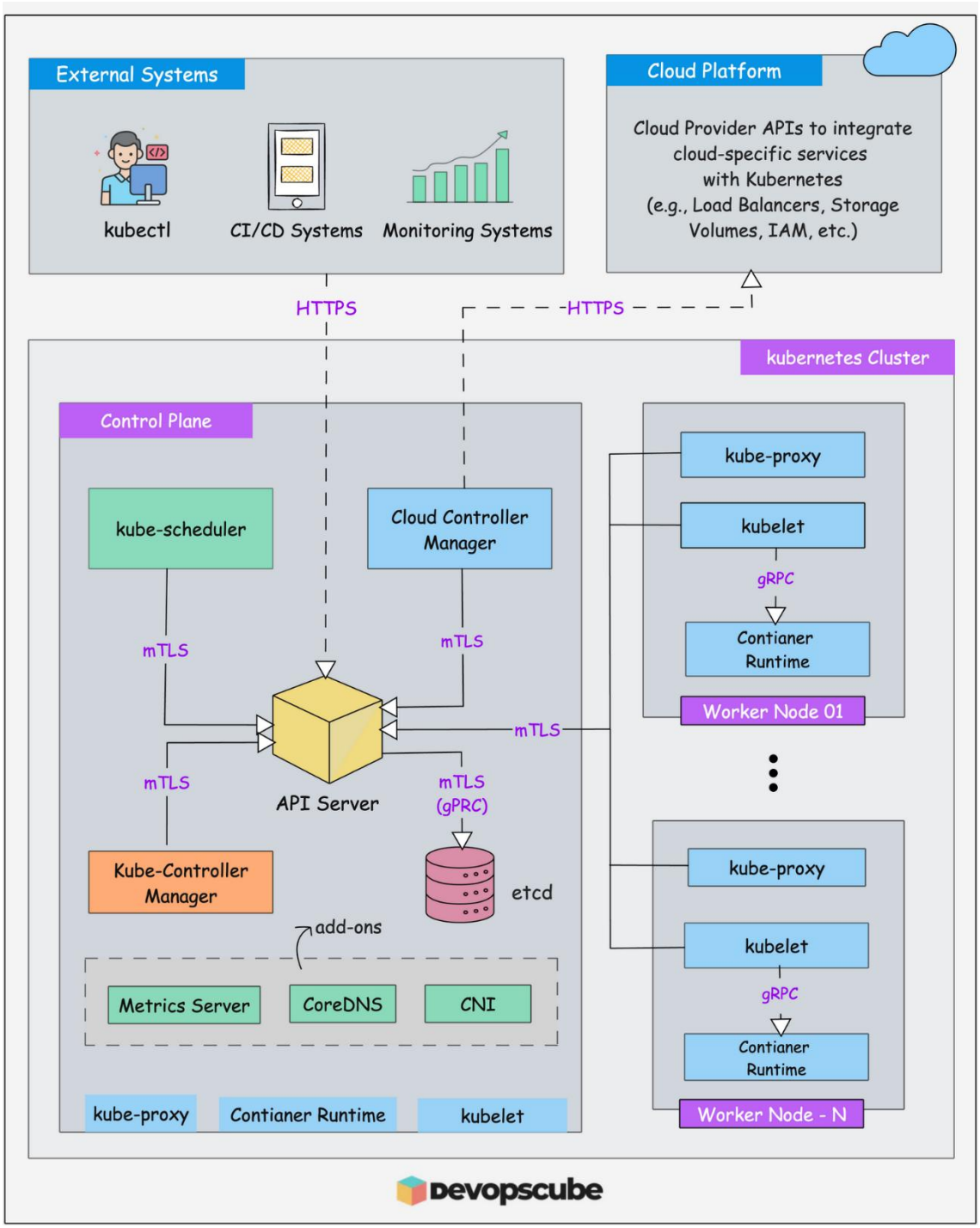
5. Actualidad [2024-2026]

Habiendo superado ya su **décimo aniversario en código abierto**, el nombre del juego ya no es solo desplegar microservicios. Kubernetes ha evolucionado para convertirse en el sustrato donde operan los grandes clústeres de **Inteligencia Artificial (IA)** y **Machine Learning**, optimizando la orquestación de GPUs a nivel global y madurando interfaces como el Gateway API y tecnologías basadas en eBPF. [1]

¿Por qué se llama K8s?

La abreviación **K8s** es un numerónimo. Proviene simplemente de contar las **8 letras** que se encuentran entre la letra **K** inicial y la **s** final de la palabra *K-u-b-e-r-n-e-t-e-s*. En griego clásico, la palabra significa *"timonel"* o *"piloto de barco"*, razón por la cual su logo es la rueda de un timón. [1, 2, 3]

Descripción del cluster K8s



Para que un clúster de Kubernetes funcione, se divide siempre en dos grandes grupos de componentes: el Plano de Control (*Control Plane*), que toma las decisiones globales y actúa como el cerebro, y los Nodos de Trabajo (*Worker Nodes*), que son las máquinas encargadas de ejecutar físicamente tus aplicaciones.

Adicionalmente, kubectl es la herramienta oficial de línea de comandos (CLI) para interactuar con un clúster de Kubernetes. Es el control remoto con el que los desarrolladores y administradores le envían órdenes al clúster desde su propia computadora.

1. El Plano de Control (Control Plane)

Es el conjunto de componentes que controlan el clúster. Por lo general, se ejecutan en servidores dedicados y altamente protegidos.

- **kube-apiserver (La Médula Espinal):** Es el **único punto de entrada al clúster** y expone la API REST de Kubernetes. Se encarga de autenticar, autorizar y validar todas las peticiones (ya vengan de kubectl o de componentes internos) antes de guardarlas.
- **etcd (El Disco Duro):** Es una base de datos clave-valor, distribuida y consistente. **Guarda absolutamente todo el estado del clúster** (configuraciones, secretos, qué está corriendo y qué no). Ningún componente habla con etcd directamente, excepto el API Server.
- **kube-scheduler (El Asignador):** Es el encargado de **elegir en qué nodo se va a ejecutar cada Pod nuevo**. Evalúa la disponibilidad de hardware (CPU/RAM), las restricciones de red y las políticas de afinidad para tomar la decisión.
- **kube-controller-manager (El Termostato):** Ejecuta de forma empaquetada todos los bucles de control internos del clúster. Su misión es **comparar el estado real con el deseado** y corregir los desvíos (por ejemplo, si un nodo falla, manda a recrear los Pods perdidos).
- **cloud-controller-manager (El Enlace Cloud):** Separa la lógica específica de los proveedores de nube (AWS, GCP, Azure) del código nativo de Kubernetes. Permite gestionar de forma automática recursos externos como balanceadores de carga o discos de almacenamiento en la nube.

2. Los Nodos de Trabajo (Worker Nodes)

Son las máquinas (físicas o virtuales) donde se ejecutan los contenedores de tus aplicaciones. Tienen tres componentes esenciales:

- **kubelet (El Capataz):** Es el agente que se ejecuta en cada nodo. **Recibe las órdenes del API Server y se asegura de que los contenedores estén**

corriendo y saludables. Si un contenedor se cae en su máquina, él lo reinicia.

- **kube-proxy (El Policía de Tránsito):** Es el administrador de red de cada nodo. **Mantiene las reglas de red en la máquina** (usando herramientas del kernel de Linux como iptables o IPVS) para que el tráfico llegue correctamente a los Pods a través de los *Services*.
- **Container Runtime (El Motor):** Es el software encargado de ejecutar los contenedores de forma real (descargar la imagen, aislar los recursos, etc.). Kubernetes se conecta a él mediante la interfaz CRI. Los motores más comunes actualmente son **containerd** y **CRI-O**.

3. La herramienta cliente (Kubectl)

kubectl es la herramienta oficial de línea de comandos (CLI) para interactuar con un clúster de Kubernetes. Es el control remoto con el que los desarrolladores y administradores le envían órdenes al clúster desde su propia computadora.

¿Cómo funciona?

kubectl no ejecuta ninguna acción mágica por sí mismo. Su única tarea es **traducir tus comandos sencillos en peticiones HTTP RESTful** y enviarlas directamente al **kube-apiserver** del clúster.

Para saber a qué clúster apuntar y qué credenciales usar para autenticarse, utiliza un archivo de configuración local llamado **kubeconfig** (ubicado por defecto en ~/.kube/config).

Lo que puedes hacer con él (Los 4 pilares)

- **Crear y desplegar recursos:** Enviar manifiestos YAML al clúster para levantar aplicaciones.
- **Inspeccionar el clúster:** Ver el estado de los Pods, nodos, servicios o configuraciones en tiempo real.
- **Depurar y solucionar problemas:** Leer los registros (*logs*) de los contenedores o meterte dentro de ellos para ejecutar comandos de prueba.
- **Modificar el estado:** Escalar aplicaciones, eliminar recursos o pausar despliegues sobre la marcha.

Descripción de los componentes del controlplane

kube-apiserver

El **kube-apiserver** es la **puerta de entrada principal y el único centro de comunicaciones** de Kubernetes. Si etcd es el disco duro y los controladores son el cerebro, el API-Server es la médula espinal: absolutamente todo lo que entra, sale o se mueve dentro del clúster pasa obligatoriamente por él. [1, 2, 3, 4, 5]

Expone una interfaz RESTful HTTP que traduce cualquier orden (ya venga de kubectl, de un dashboard, de los nodos o del plano de control) en acciones dentro del clúster. [1, 2]

El "Filtro de Seguridad": ¿Qué pasa cuando envías una petición?

Cada vez que ejecutas un comando como kubectl apply -f pod.yaml, el API-Server procesa la solicitud a través de **4 fases estrictas y secuenciales** antes de que ocurra cualquier cambio: [1, 2]

[Petición HTTP] —► 1. Autenticación —► 2. Autorización —► 3. Mutating Webhooks —► 4. Validación —► [Guardar en etcd]

1. Autenticación (Authentication)

- **¿Qué hace?:** Comprueba **quién eres**.
- **Cómo funciona:** Revisa si la petición incluye un certificado digital válido, un token de cuenta de servicio (*Service Account*) o credenciales OpenID. Si no te reconoce, rechaza la conexión de inmediato (Error 401 Unauthorized).

2. Autorización (Authorization)

- **¿Qué hace?:** Comprueba **qué tienes permitido hacer**.
- **Cómo funciona:** Una vez que sabe quién eres, evalúa las reglas **RBAC** (*Role-Based Access Control*). El hecho de que seas un usuario válido no significa que tengas permiso para crear recursos en un *namespace* de producción. Si no tienes permisos, la petición es bloqueada (Error 403 Forbidden).

3. Control de Admisión Mutante (Mutating Admission Webhooks)

- **¿Qué hace?:** Puede **modificar tu petición** sobre la marcha.
- **Cómo funciona:** Antes de evaluar si el archivo YAML está bien escrito, Kubernetes puede pasar la solicitud por herramientas externas (como Istio o Kyverno). Por ejemplo, si olvidaste ponerle políticas de seguridad, este

paso puede inyectar automáticamente configuraciones de red o contenedores *sidecar* a tu Pod.

4. Validación de Admisión (Validating Admission & Schema Validation)

- **¿Qué hace?:** Comprueba que **la petición tenga sentido y sea correcta**.
- **Cómo funciona:** El API-Server verifica la sintaxis del archivo (que los campos existan en Kubernetes) y ejecuta reglas de negocio estrictas. Si intentas pedir 500 CPUs en un nodo que solo tiene 4, o si el formato del JSON/YAML está roto, el API-Server rechazará la solicitud aquí. [1]

Solo si supera estas 4 fases con éxito, el API-Server procede a escribir la información en la base de datos **etcd**. [1, 2]

Sus dos superpoderes operativos

1. Es el único que toca la base de datos (Gobernanza)

Como se mencionó al hablar de etcd, **ningún otro componente de Kubernetes tiene permitido escribir o leer directamente en la base de datos**. El API-Server es el único intermediario legal. Esto asegura que el estado del clúster jamás se corrompa, ya que toda nueva información pasa primero por sus filtros de validación y seguridad. [1, 2]

2. El Mecanismo de Observación (Watch API)

En lugar de que los componentes como el *Scheduler* o el *Controller Manager* le pregunten al API-Server cada 3 segundos: *"¿Hay trabajo nuevo para mí?"* (lo cual saturaría el servidor), el API-Server implementa conexiones HTTP persistentes mediante un patrón llamado **Watch**. [1]

- Los componentes se "suscriben" a flujos de eventos específicos.
- En el momento en que cambia algo en etcd, el API-Server **notifica proactivamente** a los interesados. *(Ej: Le avisa al Scheduler: "Oye, se acaba de guardar un Pod nuevo que no tiene nodo asignado, calcúlalo")*. [1, 2]

Diseñado para escalar horizontalmente

El API-Server es **completamente sin estado (stateless)**. No guarda datos en su propia memoria, todo lo saca de etcd. [1, 2]. Esto significa que si tu clúster empieza a recibir millones de peticiones, puedes clonar y levantar **2, 5 o 10 instancias del kube-apiserver** detrás de un balanceador de carga estándar para repartir el tráfico sin riesgo de que se confundan o pierdan sincronización. [1, 2]

etcd

El **etcd** es la base de datos oficial, única y central de Kubernetes. Si el clúster fuera una computadora, etcd sería el disco duro que guarda todo el sistema operativo; si se corrompe o se pierde, Kubernetes olvida instantáneamente qué aplicaciones tiene instaladas y cómo debe funcionar. [1]

Técnicamente, es un almacén de datos distribuido de tipo **clave-valor** (*key-value store*) diseñado para ser extremadamente rápido, ligero y, sobre todo, consistente. [1, 2]

¿Qué información guarda exactamente?

Absolutamente todo lo que define tu clúster de Kubernetes se almacena en etcd en forma de texto plano o JSON: [1, 2]

- **El estado deseado:** Cuántos Pods pediste, qué imágenes de contenedor deben usar y qué puertos tienen abiertos.
- **El estado real:** Qué Pods están corriendo actualmente, en qué IP exacta se encuentran y en qué nodos de trabajo se asignaron.
- **Configuraciones y secretos:** Los *ConfigMaps*, las contraseñas, los tokens de acceso y los certificados (*Secrets*) de todo el sistema.
- **Roles de seguridad:** Todos los permisos, usuarios y cuentas de servicio (*RBAC*). [1, 2]

Las 3 reglas de oro de etcd en K8s

1. Nadie habla con etcd, excepto el API Server

Ningún componente de Kubernetes (ni el Scheduler, ni el Controller Manager, ni tus propios Pods) puede escribir o leer directamente en etcd. El único componente autorizado es el **kube-apiserver**. El API Server actúa como un guardián ("un intermediario") que valida que cualquier petición sea segura y correcta antes de guardarla en la base de datos. [1]

2. Nunca actualiza datos, siempre añade (*Append-only*)

Para garantizar la velocidad y evitar errores de corrupción, etcd no sobrescribe la información existente. Cada cambio, actualización o eliminación en el clúster se guarda como un evento nuevo al final del archivo de registro (*WAL - Write-Ahead Log*). [1]

3. Consistencia estricta sobre disponibilidad (Algoritmo Raft)

etcd prioriza que todos los nodos de la base de datos tengan **exactamente la misma información al mismo tiempo** (consistencia fuerte). Para lograr esto en entornos de alta disponibilidad, utiliza un algoritmo llamado **Raft**. Debido a cómo funciona matemática y lógicamente este algoritmo, los clústeres de etcd se configuran siempre en **números impares** de nodos (habitualmente 3 o 5 nodos) para poder elegir un "líder" y tolerar fallos sin perder el consenso. [1, 2]

¿Por qué es tan crítico cuidarlo?

Si tu aplicación de base de datos habitual (como MySQL o PostgreSQL) deja de funcionar, tu aplicación web fallará, pero el clúster de Kubernetes seguirá vivo. En cambio, **si etcd se cae, Kubernetes queda paralizado**: no se pueden crear nuevos Pods, no se pueden procesar despliegues, no se pueden leer secretos y el plano de control colapsa por completo. Por este motivo, hacer **backups periódicos de las snapshots de etcd** es la tarea de administración más importante en Kubernetes. [1, 2]

kube-scheduler

El **kube-scheduler** es el componente del plano de control (*control plane*) de Kubernetes que actúa como un **"oficinista de asignaciones"**. Su única y exclusiva tarea es decidir en qué nodo de trabajo (*Worker Node*) debe ejecutarse cada Pod recién creado. [1, 2, 3]

Una vez que toma la decisión, escribe el nombre del nodo elegido en la especificación del Pod y delega la ejecución real al kubelet de ese nodo. El planificador toma esta decisión **una sola vez por Pod** y no vuelve a vigilarlo a menos que el Pod sea eliminado y recreado. [1]

¿Cómo decide? El proceso de 2 pasos

Para encontrar el hogar perfecto para un Pod, el kube-scheduler ejecuta un proceso de evaluación en dos fases consecutivas: [1]

1. Filtrado (*Filtering* / *Predicates*)

El planificador revisa todos los nodos disponibles en el clúster y descarta aquellos que no cumplen con los requisitos mínimos del Pod. Los nodos que pasan esta prueba se denominan *nodos viables*. Evalúa filtros como: [1, 2]

- **Recursos suficientes:** ¿Tiene el nodo suficiente CPU y memoria RAM libre para lo que pide el Pod?
- **Puertos de red:** ¿El puerto que requiere el contenedor ya está ocupado en ese nodo?

- **Tolerancias (*Tolerations*):** ¿El nodo tiene alguna restricción (*Taint*) que el Pod no pueda soportar? [1, 2]

2. Puntuación (*Scoring / Priorities*)

Con la lista de nodos viables que quedaron del paso anterior, el planificador les asigna un puntaje del 1 al 10 utilizando un conjunto de reglas de prioridad. El nodo con la **puntuación más alta** es el elegido. Evalúa criterios como: [1]

- **Estrategia de dispersión:** Por defecto, prefiere distribuir los Pods en diferentes nodos o zonas de disponibilidad para garantizar la resiliencia en caso de fallos.
- **Afinidad (*Affinity*):** ¿El Pod prefiere estar cerca de otros Pods específicos o en un tipo de hardware concreto (ej. nodos con GPU)?
- **Optimización de recursos:** Qué tan balanceado quedará el consumo de CPU/RAM del nodo si se le añade este nuevo Pod. [1, 2, 3]

El Scheduler no arranca los contenedores

Un error común es pensar que el *scheduler* mueve o levanta los contenedores. Su trabajo es **puramente intelectual**: [1]

1. El usuario pide un Pod.
2. El **Scheduler** analiza el clúster, elige el Nodo-02 y guarda esa decisión en etcd a través del API Server.
3. El agente de ese nodo específico (el kubelet) se da cuenta de la asignación y se encarga de descargar la imagen y encender los contenedores. [1, 2, 3, 4]

kube-controller-manager

El **kube-controller-manager** es un componente del plano de control (*control plane*) de Kubernetes que actúa como un **cerebro regulador**: ejecuta bucles de control continuos para comparar el estado actual del clúster con el estado deseado y tomar acciones correctivas automáticas. [1, 2, 3]

¿Cómo funciona? (El principio del termostato)

Funciona de forma similar a un termostato en una casa: [1]

1. **Lee** el estado deseado desde los manifiestos (por ejemplo, *"quiero 3 réplicas de este Pod"*).
2. **Observa** el estado real del clúster consultando al servidor API (*API Server*).

3. **Compara** ambas realidades. Si hay una diferencia (por ejemplo, un Pod se cayó y ahora solo hay 2 réplicas), el controlador actúa ordenando la creación o eliminación de recursos para **conciliar** el sistema. [1, 2, 3]

Principales controladores incluidos

El binario del kube-controller-manager agrupa múltiples controladores internos que gestionan diferentes aspectos del clúster: [1, 2]

- **Controlador de Replicación (*Replication Controller / ReplicaSet*):** Garantiza que el número exacto de réplicas de un Pod esté siempre en ejecución. [1, 2]
- **Controlador de Nodos (*Node Controller*):** Vigila el estado de los nodos y responde si uno deja de emitir señales de vida (*heartbeats*), marcándolo como no disponible y reubicando sus cargas.
- **Controlador de Puntos de Finalización (*Endpoints Controller*):** Conecta los *Pods* con los *Services* actualizando las direcciones IP de destino. [1]
- **Controlador de Cuentas de Servicio (*Service Account Controller*):** Crea las cuentas de servicio y los tokens de acceso predeterminados para los nuevos espacios de nombres (*namespaces*). [1]
- **Controlador de Trabajos (*Job Controller*):** Supervisa tareas por lotes (*Jobs*) para asegurar que se ejecuten hasta completarse.

Variantes importantes

- **cloud-controller-manager:** Es una versión separada que interactúa específicamente con las APIs de proveedores de nube (como AWS, GCP o Azure) para gestionar recursos externos, tales como balanceadores de carga de red o volúmenes de almacenamiento persistente. [1, 2]

Descripción de los componentes de los worker

kubelet

El **kubelet** es el **"capataz de obra"** de Kubernetes. Es el componente más importante que se ejecuta en cada uno de los nodos de trabajo (*Worker Nodes*) y su función principal es garantizar que los contenedores especificados en los Pods estén realmente corriendo y saludables.

A diferencia de los componentes anteriores (etcd, scheduler, controller-manager), que viven en el plano de control central, el kubelet vive en la línea de fuego: **directamente en las máquinas donde corren tus aplicaciones.**

Las tareas principales del Kubelet

- **Recibir órdenes del plano de control:** El kubelet vigila constantemente el kube-apiserver (o lee archivos locales) para ver si se le ha asignado un Pod a su nodo.
- **Interactuar con el motor de contenedores (CRI):** El kubelet no sabe cómo crear un contenedor por sí mismo. En su lugar, le da órdenes al motor de contenedores instalado en la máquina (como **containerd** o **CRI-O**) utilizando una interfaz estándar llamada *Container Runtime Interface*. Le dice cosas como: *"Descarga esta imagen y enciende este contenedor con estos límites de memoria"*.
- **Monitorear la salud (Probes):** Ejecuta continuamente las pruebas de vida que hayas configurado en tus manifiestos:
    - **Liveness Probes:** Revisa si el contenedor sigue vivo (si falla, el kubelet lo reinicia).
    - **Readiness Probes:** Revisa si la aplicación ya está lista para recibir tráfico de usuarios.
- **Reportar el estado del nodo:** Envía actualizaciones constantes (*heartbeats*) al API Server para avisar si el nodo tiene suficiente memoria/CPU o si está experimentando problemas, permitiendo que el *Scheduler* sepa si puede enviarle más trabajo.

La regla de oro del Kubelet: Solo le importan sus Pods

El kubelet es completamente agnóstico al resto del clúster. No sabe qué es un *Service*, no entiende de *Ingress* ni le interesa qué está pasando en los nodos vecinos. **Su único universo son los Pods que el API Server le asignó a su propia máquina**.

Si un Pod asignado a su nodo se cae por un error interno del contenedor, el kubelet lo reiniciará inmediatamente para intentar volver al estado deseado. Sin embargo, si la máquina entera (el nodo) se apaga o se rompe físicamente, el kubelet muere con ella; en ese caso, será el *Controller Manager* (desde el plano de control central) quien note su ausencia y mande a recrear esos Pods en otro nodo vivo.

El "Modo Autónomo" (Static Pods)

Aunque normalmente recibe órdenes del API Server, el kubelet puede funcionar de manera independiente. Si colocas un archivo de manifiesto YAML en una carpeta específica del disco duro del nodo (por defecto /etc/kubernetes/manifests), el kubelet lo leerá y levantará los contenedores de forma autónoma.

A estos se les conoce como **Static Pods** y es, de hecho, la forma en que muchas herramientas (como kubeadm) levantan los propios componentes de Kubernetes (como el API Server o etcd) al inicializar un clúster.

kube-proxy

El **kube-proxy** es el **"policía de tránsito"** o el **administrador de red** de Kubernetes. Es un componente esencial que se ejecuta en cada nodo del clúster (tanto de control como de trabajo) y su única función es **gestionar las reglas de red para permitir la comunicación hacia tus Pods**.

Específicamente, el kube-proxy se encarga de que los **Services** de Kubernetes funcionen. Cuando creas un Service (que tiene una IP virtual única y estable), el kube-proxy se asegura de que cualquier tráfico enviado a esa IP sea redirigido correctamente a los Pods reales que están detrás de ella.

¿Qué hace exactamente?

- **Vigila el API Server:** Está constantemente escuchando al kube-apiserver. Cada vez que creas, modificas o eliminas un *Service* o un *Endpoint* (las IPs de los Pods asociados), el kube-proxy se entera de inmediato.
- **Traduce IPs Virtuales a IPs Reales:** Los *Services* tienen IPs que no existen físicamente en ninguna tarjeta de red (ClusterIP). El kube-proxy escribe reglas en el sistema operativo del nodo para interceptar el tráfico dirigido a esa IP virtual y desviarlo hacia la IP real de uno de los Pods.
- **Balanceo de carga básico:** Si un Service tiene 3 Pods por detrás, el kube-proxy se encarga de repartir el tráfico entre ellos de forma equitativa (habitualmente de manera aleatoria o por turnos).

¿Cómo lo hace por dentro? (Modos de operación)

El kube-proxy no procesa el tráfico de red paquete por paquete (eso sería muy lento). En su lugar, **configura las herramientas de red nativas del kernel de Linux** de la máquina. Dependiendo de cómo esté configurado tu clúster, utiliza uno de estos modos:

1. **IPVS (IP Virtual Server):** Es el modo recomendado para clústeres medianos y grandes. Utiliza el balanceador de carga integrado en el kernel de Linux. Es extremadamente rápido (escala de manera eficiente con miles de servicios) y ofrece algoritmos avanzados de balanceo.
2. **iptables:** Es el modo tradicional por defecto. El kube-proxy crea una lista enorme de reglas de firewall en el nodo. Funciona muy bien en clústeres pequeños, pero si tienes miles de servicios, el kernel de Linux puede volverse lento al procesar tantas reglas secuenciales.
3. **NFTables:** El reemplazo moderno de iptables en distribuciones de Linux recientes, adoptado de manera estable en las versiones de Kubernetes de los últimos años [2024-2026] para mejorar el rendimiento.
4. **Userspace (Obsoleto):** Un modo antiguo donde el tráfico pasaba físicamente por el proceso del kube-proxy. Ya no se utiliza debido a su alto impacto en el rendimiento.

Una aclaración importante: kube-proxy vs CNI (Calico, Flannel, Cilium)

Es común confundir el kube-proxy con el plugin de red (CNI) del clúster, pero hacen cosas distintas:

- El **CNI** (como Calico o Cilium) se encarga de la **red básica**: le asigna una IP única a cada Pod y permite que el Pod del Nodo A pueda hablar directamente con el Pod del Nodo B.
- El **kube-proxy** se monta encima de esa red y se encarga únicamente de las **abstracciones (Services)**: mapear la IP estática del Service hacia las IPs cambiantes de los Pods.

Nota moderna: En la actualidad, herramientas avanzadas de CNI basadas en eBPF (como **Cilium**) permiten desactivar por completo el kube-proxy (kube-proxyless mode), ya que el propio plugin de red asume la gestión de los Services de una forma mucho más eficiente.

Container Runtimes en Kubernetes

En la actualidad, Kubernetes no interactúa directamente con los contenedores, sino que lo hace a través de una interfaz estándar llamada CRI (Container Runtime Interface). Esto significa que el clúster delegará la creación, ejecución y destrucción de los contenedores a un motor especializado.

Tras la histórica deprecación y eliminación definitiva del soporte directo a Docker (conocido como *Docker-shim*), el ecosistema se consolidó en torno a opciones más ligeras, rápidas y diseñadas específicamente para Kubernetes. Los motores de contenedores más utilizados son:

1. containerd

Es el **líder absoluto del mercado** y el motor por defecto en la gran mayoría de las distribuciones y servicios administrados de Kubernetes (como EKS de AWS, GKE de Google Cloud y AKS de Azure).

- **Origen:** Nació originalmente como un componente interno de Docker, pero fue donado a la CNCF (donde hoy es un proyecto graduado) para convertirse en un motor independiente y puramente enfocado en la eficiencia.
- **Características:** Es extremadamente estable, maduro y consume muy pocos recursos de memoria y CPU. Se encarga de todo el ciclo de vida del contenedor (transferencia de imágenes, ejecución, almacenamiento y gestión de red de bajo nivel).

2. CRI-O

Es la principal **alternativa nativa de código abierto** a containerd, desarrollada con una filosofía minimalista bajo el paraguas de Red Hat.

- **Origen:** Fue creado desde cero con un único propósito en mente: servir a la interfaz CRI de Kubernetes. Su lema es *"nacer para servir a Kubernetes, y a nadie más"*. Es el motor por defecto en plataformas corporativas como **Red Hat OpenShift**.
- **Características:** Es increíblemente ligero porque no incluye ninguna característica que Kubernetes no pida explícitamente. Cumple estrictamente con los estándares de la OCI (Open Container Initiative), lo que garantiza que cualquier imagen creada con Docker correrá perfectamente en él.

3. Motores de Aislamiento Avanzado (Sandboxed Runtimes)

Para entornos de alta seguridad (como sistemas bancarios o plataformas multi-inquilino) donde los contenedores estándar de Linux no ofrecen suficiente aislamiento, se suelen integrar motores de "caja de arena" mediante la especificación RuntimeClass de Kubernetes:

- **gVisor:** Creado por Google, intercepta las llamadas al sistema (*syscalls*) del contenedor y las ejecuta en un kernel virtual en el espacio de usuario, impidiendo que un contenedor comprometido dañe al servidor anfitrión.
- **Kata Containers:** Ejecuta cada Pod dentro de una micro-máquina virtual ultra-rápida y ligera. Combina la seguridad extrema de una máquina virtual tradicional con la velocidad de despliegue de un contenedor.

Comparativa Directa

Característica	containerd	CRI-O	Kata / gVisor
Enfoque principal	Propósito general y adopción masiva	Diseñado exclusivamente para K8s	Seguridad y aislamiento extremo
Consumo de recursos	Muy bajo	Ultra bajo	Mayor (debido a la capa de seguridad)
Dónde se usa más	GKE, EKS, AKS, clústeres con kubeadm	OpenShift, entornos empresariales Red Hat	Entornos bancarios, gobierno o multi-tenant

*(Nota: Si extrañas usar comandos de Docker en tu terminal para inspeccionar estos motores en tus nodos, hoy en día se utiliza la herramienta CLI oficial llamada **crictl**, la cual permite interactuar con cualquiera de estos motores usando una sintaxis idéntica).*

Objetos y Workloads

En Kubernetes, para entender cómo funciona la plataforma, la clave está en diferenciar los **Objetos** (los bloques de construcción o "sustantivos") de las **Cargas de Trabajo** o workloads (el software corriendo que usa esos bloques para funcionar).

En términos sencillos: **un Objeto representa cualquier entidad persistente en el sistema**, mientras que **una Carga de Trabajo es un objeto específico (o conjunto de ellos) diseñado para gestionar la ejecución de tus aplicaciones**.

Objetos en Kubernetes (Kubernetes Objects)

Un objeto es un **"registro de intenciones"**. Cuando creás un objeto, le decís a Kubernetes cuál es el **estado deseado** de tu clúster (qué recursos querés, qué políticas aplicar, etc.), y el plano de control (Control Plane) trabaja continuamente para mantener ese estado.

Componentes de Red (Networking)

- **Service (Servicio):** Expone una abstracción para conectar un grupo de Pods a la red (con una IP estable y balanceo de carga), evitando que el tráfico se corte si los Pods mueren y cambian de IP.
- **Ingress:** Gestiona el acceso externo a los servicios del clúster (normalmente tráfico HTTP/HTTPS). Actúa como un proxy reverso o router avanzado.
- **IngressClass:** Define la configuración y el controlador específico (como Nginx, Traefik, Kong) que procesará los recursos Ingress.
- **Gateway / GatewayClass (Gateway API):** La evolución moderna de Ingress. Ofrece un control más fino y expresivo de la red orientado a diferentes roles de equipo (infraestructura, operaciones, desarrollo).
- **NetworkPolicy:** Define las reglas de firewall internas del clúster. Controla qué Pods pueden comunicarse entre sí y con el exterior.

Componentes de Almacenamiento (Storage)

- **PersistentVolume (PV):** Un pedazo de almacenamiento físico o en la nube aprovisionado en el clúster. Es independiente del ciclo de vida de cualquier Pod.
- **PersistentVolumeClaim (PVC):** Una solicitud de almacenamiento por parte de un usuario o Pod. Funciona como un "voucher" para consumir un PV.

- **StorageClass:** Define "perfiles" o clases de almacenamiento (ej: SSD rápido, HDD lento, almacenamiento en la nube) para que los PV se creen automáticamente bajo demanda.
- **VolumeAttachment:** Objeto interno que rastrea si un volumen de almacenamiento está conectado a un nodo específico.

Configuración y Secretos (Configuration)

- **ConfigMap:** Almacena datos de configuración no confidenciales en formato clave-valor (ej: archivos de configuración, variables de entorno) para que los usen los Pods.
- **Secret:** Similar a un ConfigMap, pero diseñado específicamente para datos sensibles (contraseñas, tokens, llaves SSH). Se guardan codificados (Base64) o encriptados en reposo.
- **ResourceQuota:** Limita el consumo total de recursos (CPU, memoria, cantidad de objetos) que puede usar un Namespace determinado.
- **LimitRange:** Define límites mínimos y máximos de recursos (CPU/Memoria) permitidos para los Pods de forma individual dentro de un Namespace.

Seguridad, Acceso y Control (Security & RBAC)

- **ServiceAccount:** Proporciona una identidad de seguridad para los procesos que corren dentro de los Pods, permitiéndoles autenticarse ante la API de Kubernetes.
- **Role / ClusterRole:** Definen permisos (qué acciones se pueden hacer sobre qué recursos). Role aplica a un Namespace; ClusterRole aplica a todo el clúster.
- **RoleBinding / ClusterRoleBinding:** Vinculan un Rol (permisos) a un usuario, grupo o ServiceAccount.
- **ValidatingWebhookConfiguration / MutatingWebhookConfiguration:** Registran servicios externos que pueden interceptar, validar o modificar las solicitudes de la API de Kubernetes antes de que se guarden.

Estructura y Metadatos (Cluster Metadata)

- **Namespace:** Divide un clúster físico en múltiples "clústeres virtuales". Es ideal para separar entornos (Desarrollo, Testing, Producción) o equipos.
- **Event:** Registros temporales generados automáticamente cuando ocurre un cambio de estado en el clúster (ej: un Pod que arranca o falla).

- **Node:** Representa una máquina física o virtual de trabajo (Worker Node) dentro del clúster.

Cargas de Trabajo en Kubernetes (Workloads)

Las cargas de trabajo son aplicaciones que se ejecutan en Kubernetes. Todas las cargas de trabajo **se ejecutan dentro de Pods**, pero usar un Pod suelto no es seguro. Por eso, se utilizan estos controladores específicos que se encargan de automatizar la resiliencia, el escalado y el ciclo de vida de los contenedores.

1. Cargas de Trabajo Sin Estado (Stateless)

- **Pod:** El objeto más pequeño y básico de Kubernetes. Representa un único proceso en ejecución. Contiene uno o más contenedores estrechamente acoplados que comparten red y almacenamiento.
- **ReplicaSet:** Su único trabajo es garantizar que un número exacto de réplicas idénticas de un Pod estén corriendo en cualquier momento. Si un Pod muere, levanta otro.
- **Deployment:** La carga de trabajo más común. Administra ReplicaSets de forma declarativa. Permite actualizar tus aplicaciones sin cortes de servicio (Rolling Updates), hacer play/pausa en despliegues y volver a versiones anteriores (Rollbacks).

2. Cargas de Trabajo Con Estado (Stateful)

- **StatefulSet:** Diseñado para aplicaciones que necesitan mantener una identidad persistente y única. A diferencia de un Deployment (donde los Pods son intercambiables), cada Pod de un StatefulSet tiene un identificador único (ej: db-0, db-1), mantiene su propio disco asignado aunque se mueva de nodo, y se despliega en un orden estricto. Ideal para bases de datos (PostgreSQL, MongoDB) o clústeres de mensajería (Kafka).

3. Tareas Temporales o Programadas (Batch)

- **Job:** Crea uno o más Pods y se asegura de que terminen con éxito una tarea específica (ej: procesar un lote de imágenes o migrar una base de datos). Una vez que la tarea termina, los contenedores se apagan y no se reinician.
- **CronJob:** Ejecuta Jobs de manera periódica basándose en una programación temporal al estilo Linux (ej: realizar un backup todas las noches a las 3:00 AM).

4. Infraestructura y Demonios del Nodo (System)

- **DaemonSet:** Garantiza que **todos** (o algunos) los nodos del clúster ejecuten una copia idéntica de un Pod. Si agregás un nodo nuevo al clúster,

Kubernetes le crea un Pod del DaemonSet automáticamente. Se usa casi exclusivamente para herramientas de infraestructura (colectores de logs como Fluentd, agentes de monitoreo como Prometheus Node Exporter, o proxies de red).

Primeros pasos con kubernetes y kubectl

Para desplegar entornos de pruebas y laboratorios de Kubernetes en tu máquina local o en pipelines de integración continua, Kind, Minikube y K3s son las tres herramientas líderes, cada una diseñada con un enfoque técnico y un propósito arquitectónico radicalmente distintos.

A continuación, se presenta una guía exhaustiva de su implementación, seguida de una tabla comparativa detallada para ayudarte a elegir la opción adecuada según tus necesidades de desarrollo. [1]

Guía de Implementación Básica

Para realizar los siguientes despliegues, es necesario contar con un motor de contenedores instalado (como **Docker** o **Podman**) y la herramienta de línea de comandos kubectl.

Como prerrequisito, cumplir con requerimientos de hardware:

- 2 CPUs o más
- 4GB o más de memoria libre
- 20GB o más de espacio libre
- Conexión a Internet
- Container runtimes: Docker, Podman, etc

Instalar la herramienta kubectl:

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"

sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

kubectl version --client
```

1. Kind (Kubernetes in Docker)

Kind emula los nodos del clúster ejecutando contenedores Docker como si fueran máquinas físicas o virtuales. Es la herramienta estándar para las pruebas internas del propio equipo de Kubernetes. [1, 2]

- **Instalación rápida:**

```
[ $(uname -m) = x86_64 ] && curl -Lo ./kind
https://kind.sigs.k8s.io/dl/v0.33.0/kind-$(uname)-amd64

chmod +x ./kind

sudo mv ./kind /usr/local/bin/kind
```

- **Creación de un clúster estándar (un solo nodo) basado en podman:**

```
export KIND_EXPERIMENTAL_PROVIDER=podman kind create cluster --name cluster-test
```

- **Despliegue multi-nodo avanzado (mediante archivo de configuración YAML):**
  Crea un archivo llamado kind-config.yaml:

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
- role: worker
- role: worker
```

Y ejecuta:

```
export KIND_EXPERIMENTAL_PROVIDER=podman kind create cluster --config kind-config.yaml
```

2. Minikube

Es el laboratorio local clásico de la CNCF. Funciona levantando una máquina virtual dedicada (usando VirtualBox, QEMU o Hyper-V) o directamente sobre un contenedor Docker aislado mediante su sistema de "drivers". [1, 2]

- **Instalación rápida:**

```
curl -LO https://github.com/kubernetes/minikube/releases/latest/download/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube && rm minikube-linux-amd64
```

- **Creación de un clúster especificando recursos y el driver de Docker:**

```
minikube start --drivers=podman --cpus=2 --memory=4096 --nodes=2
```

- **Habilitar extensiones (Addons):** Minikube destaca por su facilidad para encender herramientas complejas con un comando: [1]

```
minikube addons enable ingress
minikube addons enable dashboard
```

3. K3s (por Rancher)

K3s es una distribución oficial y certificada de Kubernetes sumamente ligera (pesa menos de 100 MB). Consigue esto eliminando plugins de nube heredados y reemplazando la base de datos etcd por una infraestructura SQL más liviana (SQLite) de forma predeterminada. Para entornos de prueba locales en sistemas operativos Mac o Windows, se suele implementar envuelto en la herramienta **k3d**, la cual corre K3s dentro de contenedores Docker. [1, 2, 3]

- **Instalación nativa directa (solo para Linux/WSL):**

```
curl -sfL https://get.k3s.io | sh -
```
El archivo de configuración de acceso queda en /etc/rancher/k3s/k3s.yaml. Para utilizar kubectl:

```
export KUBECONFIG=/etc/rancher/k3s/k3s.yaml
```

Tabla Comparativa Exhaustiva

Criterio Técnico	Kind (K8s in Docker)	Minikube	K3s (vía k3d)
Arquitectura de Nodos	Cada nodo es un contenedor Docker.	Un nodo en VM dedicada o contenedor.	Binario único ultra-ligero de producción.
Tiempo de Arranque	Muy rápido (**30 - 60 segundos**).	Lento (**2 - 5 minutos**) debido a la virtualización.	Ultra rápido (**20 - 40 segundos**).
Uso de Memoria Mínimo	Bajo (~500 MB por contenedor).	Alto (Mínimo **2 GB** dedicados).	Extremadamente bajo (**~512 MB**).
Clúster Multi-nodo	Excelente, nativo y muy ligero.	Soportado, pero consume muchos recursos.	Excelente (escalable a nivel de producción).
Ecosistema de Complementos	Manual (Debés instalar Ingress/Métricas por tu cuenta).	**Excelente (MÁS DE 30 ADDONS)** nativos listos para activar.	Incluye Traefik como Ingress y Local-Path Storage de fábrica.
Persistencia al Apagar	Se pierde el estado si destruís el contenedor.	Mantiene el estado nativamente al pausar la máquina (minikube stop).	Mantiene el estado en volumen si se usa k3d o servicio del sistema.
Propósito Principal	**Pipelines de CI/CD** y pruebas de conformidad de la API.	**Aprendizaje inicial**, prácticas locales y certificaciones (CKA).	**Homelabs**, entornos perimetrales (Edge Computing, IoT) e integración continua.

¿Cuál deberías elegir?

1. Elegí **Kind** si tu prioridad absoluta es la velocidad en **automatizaciones o pipelines de CI/CD**, o si necesitás simular múltiples nodos de trabajo de manera eficiente sin sobrecargar de memoria tu computadora personal. [1, 2]
2. Elegí **Minikube** si estás **empezando a estudiar Kubernetes** o preparando la certificación CKA. Contar con un catálogo de addons listos para usar te evitará dolores de cabeza configurando redes e ingresos de tráfico. [1, 2]
3. Elegí **K3s / k3d** si querés montar un entorno ligero que se comporte de forma **idéntica a un entorno real de producción** o si estás desarrollando software para dispositivos con recursos de hardware muy limitados. [1, 2]

Primeros pasos con kubectl

kubectl es la interfaz de línea de comandos (CLI) oficial para interactuar con el servidor de la API de Kubernetes. Actúa como el puente de comunicación entre el administrador o desarrollador y el plano de control del clúster. A través de él se pueden crear, inspeccionar, actualizar y eliminar recursos (como Pods, Deployments o Servicios) utilizando instrucciones directas (imperativas) o archivos de manifiesto YAML (declarativas). [1, 2, 3, 4, 5]

Por defecto, **kubectl busca sus credenciales y rutas de conexión en el archivo kubeconfig ubicado en $HOME/.kube/config**. [1, 2]

Descripción Detallada del Archivo de Configuración (kubeconfig)

Un archivo kubeconfig organiza la información de conectividad mediante tres pilares fundamentales que interactúan entre sí de forma modular: **Clústeres, Usuarios y Contextos**. [1, 2]

A continuación se detalla su anatomía estructural con un ejemplo real (archivo config):

```
apiVersion: v1
kind: Config
preferences: {}
clusters:
- name: mi-cluster-kind
  cluster:
    server: https://127.0.0.1:6443
    certificate-authority-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0t...
users:
- name: administrador-kind
  user:
    client-certificate-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0t...
    client-key-data: LS0tLS1CRUdJTiBQUklWQVRFIEtFWS0tLS0t...
contexts:
- name: entorno-desarrollo
  context:
    cluster: mi-cluster-kind
    user: administrador-kind
    namespace: default
current-context: entorno-desarrollo
```

Explicación Bloque por Bloque

1. Metadatos Base

- **apiVersion y kind**: Identifican la especificación técnica del objeto de configuración dentro del ecosistema de Kubernetes.
- **preferences**: Ajustes opcionales de comportamiento de la CLI (por ejemplo, habilitar/deshabilitar extensiones o advertencias específicas). [1]

2. Bloque clusters (¿A dónde me conecto?)

Define los servidores de Kubernetes de destino. Puede contener una lista de múltiples clústeres. [1]

- **name**: Un alias arbitrario y local para identificar el clúster (ej. mi-cluster-kind). [1]
- **server**: La URL base del Endpoint de la API de Kubernetes (dirección IP y puerto). En entornos locales con Kind o Minikube suele apuntar a localhost (127.0.0.1). [1, 2, 3]
- **certificate-authority-data**: El certificado de la Autoridad de Certificación (CA) codificado en Base64. Sirve para que tu máquina confíe en el servidor e impide ataques de intermediario (Man-in-the-Middle). *Nota: A veces se sustituye por insecure-skip-tls-verify: true en laboratorios no seguros.* [1]

3. Bloque users (¿Quién soy?)

Define las credenciales de autenticación para acceder a los clústeres. [1]

- **name**: El identificador o apodo que enlaza estas credenciales (ej. administrador-kind).
- **client-certificate-data y client-key-data**: Certificados TLS criptográficos del cliente (codificados en Base64) utilizados para validar tu identidad ante el API Server.
- *Alternativas de autenticación:* En nubes públicas (AWS, GCP, Azure), este bloque suele contener tokens OAuth dinámicos (token), autenticación básica (username/password) o llamadas a plugins externos ejecutables (exec). [1, 2, 3]

4. Bloque contexts (El Enlace)

Un contexto une un Clúster con un Usuario y, opcionalmente, un Namespace. Su propósito es permitirte saltar entre múltiples entornos de forma rápida. [1, 2, 3]

- **name**: Nombre de la combinación (ej. entorno-desarrollo).
- **cluster**: Debe coincidir exactamente con el name registrado en la sección de clústeres.
- **user**: Debe coincidir exactamente con el name registrado en la sección de usuarios.
- **namespace**: El espacio de nombres predeterminado para este contexto. Si ejecutas un comando como kubectl get pods, automáticamente buscará en este espacio asignado sin necesidad de añadir el flag -n. [1, 2, 3]

5. current-context (El Contexto Activo)

- Indica el contexto que kubectl utilizará por defecto para ejecutar cualquier acción inmediata. En el ejemplo, todo comando que lances se enviará al clúster mi-cluster-kind bajo los permisos del usuario administrador-kind. [1, 2]

Comandos Útiles para Gestionar la Configuración

Nunca se recomienda modificar el archivo kubeconfig a mano para evitar errores de sintaxis YAML. Para ello se utiliza el subcomando kubectl config: [1, 2]

- **Visualizar la configuración actual fusionada:**

```
kubectl config view
```

- **Listar todos los contextos disponibles:**

```
kubectl config get-contexts
```

- **Cambiar instantáneamente de clúster/entorno (modifica el current-context):**

```
kubectl config use-context entorno-desarrollo
```

- **Fijar un espacio de nombres por defecto de manera permanente para el contexto actual:**

```
kubectl config set-context --current --namespace=mi-aplicacion
```